

API Documentation

Overview

This API provides endpoints for user management and image processing functionalities. It supports user registration, authentication, and the extraction of header structures and EXIF metadata from uploaded images. JWT is used for secure access to protected endpoints.

Database Structure

Users Table

Column	Type	Description
email	String	Primary key. User's email (unique).
username	String	User's username (unique).
password	String	User's hashed password.
role	String	User's role; can be 'admin' or 'user'.
last_updated	DateTime	Timestamp of the last update (auto-updated).

JWT Authentication

JWT (JSON Web Token) authentication is used to secure endpoints and verify user identities. Upon successful login, a JWT is issued containing essential information about the user (such as their email and role). This token must be included in the Authorization header as a Bearer token for subsequent requests to protected endpoints.

1. Token Generation

When a user logs in successfully (by providing valid credentials like email and password), the server generates a JWT. This token is signed using a secret key known only to the server, ensuring its authenticity. The token is valid for **30 minutes**, after which the user must log in again to get a new token.

2. Token Verification

To access protected routes, the client must send the JWT in the Authorization header of each request, using the Bearer schema. The server then verifies the token's validity, checking:

1. The token has not been tampered with (signature verification).
2. The token is not expired (the `exp` claim is checked).
3. The user's role has the necessary permissions for the requested resource.

Token Expiration

The JWT includes an `exp` (expiration) claim, which is set to **30 minutes** from the moment the token is issued. After this time, the token is no longer valid, and any attempts to access protected endpoints with it will result in a 401 Unauthorized response. The user will need to log in again to receive a fresh token.

3. User Roles and Access Control

The **role** included in the JWT determines the level of access the user has to specific endpoints or actions. The role is verified by the server along with the token.

Common Roles:

- **Admin:** Can access and manage all resources.
- **User:** Can only access their own resources or limited functionalities.

API Endpoints

1. Register User

- **Endpoint:** /adduser
- **Method:** POST
- **Description:** Register a new user (Admin only).
- **Parameters:**
 - **username:** New user's username (string).
 - **email:** New user's email (string).
 - **password:** New user's password (string).
 - **role:** New user's role (default: 'user').
- **Responses:**
 - **201:** User registered successfully.

```
{  
  "message": "User registered successfully"  
}
```

- **403:** Not authorized (non-admin user).

Example Request

Curl

```
curl -X 'POST' \
'http://127.0.0.1:8000/adduser?username=test2&email=test2%40gmail.com&password=test123&role=user' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbkBNbWVpbC5jb20iLCJyb2x1IjoieWRtaW4iLCJleHAiOjE3OTYxNj19.DdKFmupVo5yySxaejVA2I77P9cRxofnt09tuWXRkM3Q' \
-d ''
```

Request URL

```
http://127.0.0.1:8000/adduser?username=test2&email=test2%40gmail.com&password=test123&role=user
```

Server response

Server response	
Code	Details
200	Response body <pre>{ "message": "User registered successfully" }</pre> Response headers <pre>content-length: 42</pre>

2. User Login

- **Endpoint:** /login
- **Method:** POST
- **Description:** Authenticate a user and return a JWT token.
- **Parameters:**
 - email: User's email (string).
 - password: User's password (string).
- **Responses:**
 - **200:** Access token generated successfully.

```
{  
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbkBobWFpbC5jb20iLCJyb2x1IjoieWwRtaW4iLCJleHAiOiJlE3Mjc3OTYxNj19.DdKFmupVo5yySxaejVA2I77P9cRxofnt09tuWXRkM3Q",  
  "token_type": "bearer"  
}
```

- **400:** Invalid credentials.

Example Request

Curl

```
curl -X 'POST' \
'http://127.0.0.1:8000/login?email=admin%40gmail.com&password=admin123' \
-H 'accept: application/json' \
-d ''
```

Request URL

```
http://127.0.0.1:8000/login?email=admin%40gmail.com&password=admin123
```

Server response

Code	Details
200	Response body <pre>{ "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbk8nbWZpbC5jb20iLCJyb2x1IjoiaWRTaW4iLCJleHAiOjE3Mjc3OTYxNj19.DdKFmupVo5yySxaejVA2I", "token_type": "bearer" }</pre> Response headers <pre>content-length: 198 content-type: application/json date: Tue, 01 Oct 2024 14:52:48 GMT server: uvicorn</pre>

3. Get All Users

- **Endpoint:** /getallusers
- **Method:** GET
- **Description:** Retrieve a list of all registered users (Admin only).
- **Responses:**
 - **200:** List of users returned successfully.

```
[
  {
    "email": "admin@gmail.com",
    "username": "admin",
    "password": "$2b$12$mrFgnJWjZ7qU5m0M3OSzVOWXfBhZVFNgSb3PLLCQ81q5y33RBxA.",
    "role": "admin",
    "last_updated": "2024-10-01T12:00:16.597219"
  },
  {
    "email": "user@gmail.com",
    "username": "usertest",
    "password": "$2b$12$yMQb3.OftWFFf4bH0QEIWaebZeZ/G5inywuQ9Cf3xQzdM55hZrXNta",
    "role": "user",
    "last_updated": "2024-10-01T12:00:45.235935"
  }
]
```

- **403:** Not authorized (non-admin user).

Example Request

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/getallusers' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pYk9kbWpC5jb20iLCJyb2x1IjoieYWRtaW4iLCJleHAiOjE3Mjc3OTQ3MjR9.jemqq0kbcPbiW6s0lB_v8JcADf0qMlEOaABcKw80CBI'
```

Request URL

```
http://127.0.0.1:8000/getallusers
```

Server response

Code	Details
200	Response body <pre>[{ "email": "admin@gmail.com", "username": "admin", "password": "\$2b\$12\$mrFgnJWjZ7qU5m0M30SzVOMXf8hZfVFNgSb3PLLCQ81q5y33RBxA.", "role": "admin", "last_updated": "2024-10-01T12:00:16.597219" }, { "email": "user@gmail.com", "username": "usertest", "password": "\$2b\$12\$yMQb3.OftWfF4bH0QEINaebZeZ/G5inywuQ9Cf3xQzdM55hZrXNta", "role": "user", "last_updated": "2024-10-01T12:00:45.235935" }]</pre> Response headers

4. Delete User

- **Endpoint:** /delete/{email}
- **Method:** DELETE
- **Description:** Delete a user by their email (Admin only).
- **Responses:**
 - **200:** User deleted successfully.

```
{  
  "detail": "User deleted"  
}
```

- **403:** Not authorized (non-admin user).
- **404:** User not found.

```
{  
  "detail": "User not found"  
}
```

Example Request

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/delete/test2%40gmail.com' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pYk9kbWpC5jb20iLCJyb2x1IjoieYWRtaW4iLCJleHAiOjE3Mjc3OTYxNj19.DdKFmupVo5yySxaejVA2I77P9cRxofnt09tuWXRkM3Q'
```

Request URL

```
http://127.0.0.1:8000/delete/test2%40gmail.com
```

Server response

Server response	
Code	Details
200	Response body<pre>{ "detail": "User deleted" }</pre> Response headers<pre>content-length: 25 content-type: application/json date: Tue, 01 Oct 2024 15:20:11 GMT</pre>

5. Edit User

- **Endpoint:** /edit/{email}
- **Method:** PUT
- **Description:** Edit a user's details (Admin only).
- **Parameters:**
 - new_username: New username (optional).
 - new_role: New role (optional).
- **Responses:**
 - **200:** User details updated successfully.

```
{
  "password": "$2b$12$TId18RBH/WIC7V07W4IK4ey3EpPeDmW3Y8mLANM2H3xZHqFRIapS6",
  "last_updated": "2024-10-01T14:30:01.604486",
  "email": "test@gmail.com",
  "role": "user",
  "username": "testnew"
}
```

- **403:** Not authorized (non-admin user).
- **404:** User not found.

```
{
  "detail": "User not found"
}
```

Example Request

Curl

```
curl -X 'PUT' \
'http://127.0.0.1:8000/edit/test%40gmail.com?new_username=testnew&new_role=user' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pYk9kbBnbWFpbC5jb20iLCJyb2x1IjoieYWRtaW4iLCJleHAiOjE3Mjc3OTgxNjh9.ZbsXENp5-KDSy1-Nf8eqBkEnD0-rEh0u293XciB4_t4'
```

Request URL

```
http://127.0.0.1:8000/edit/test%40gmail.com?new_username=testnew&new_role=user
```

Server response

Server response	
Code	Details
200	Response body<pre>{ "password": "\$2b\$12\$TId18RBH/WIC7V07W4IK4ey3EpPeDmW3Y8mLANM2H3xZHqFRlApS6", "last_updated": "2024-10-01T14:30:01.604486", "email": "test@gmail.com", "role": "user", "username": "testnew" }</pre> Response headers<pre>content-length: 179 content-type: application/json</pre>

6. Reset User Password

- **Endpoint:** `/reset-password/{email}`
- **Method:** PUT
- **Description:** Reset a user's password (Admin only).
- **Parameters:**
 - `new_password`: New password (string).
- **Responses:**
 - **200:** Password updated successfully.

```
{  
  "detail": "Password updated"  
}
```

- **403:** Not authorized (non-admin user).
- **404:** User not found.

```
{  
  "detail": "User not found"  
}
```

Example Request

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/reset-
password/test%40gmail.com?new_password=test1234' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1p
bkBnbWFpbC5jb20iLCJyb2x1IjoieYWRtaW4iLCJleHAiOjE3Mjc3O
TQ3MjR9.jemqq0kbcPbiW6s0lB_v8JcADf0qMlEOaABcKw80CBI'
```

Request URL

```
http://127.0.0.1:8000/reset-
password/test%40gmail.com?new_password=test1234
```

Server response

Server response	
Code	Details
200	Response body <pre>{ "detail": "Password updated" }</pre> Response headers <pre>content-length: 29</pre>

7. Extract Header Structure

- **Endpoint:** /headerstructure
- **Method:** POST
- **Description:** Upload an image and extract its header structure in HTML format.
- **Parameters:**
 - **image:** Image file to upload (multipart form data).
- **Responses:**
 - **200:** Header structure extracted successfully.

```
{  
  "html_content": "<html>...</html>"  
}
```

- **404:** No header structure found.
- **500:** Internal server error.

Example Request

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/headerstructure' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'image=@test1.png;type=image/png'
```

Request URL

http://127.0.0.1:8000/headerstructure

Server response

Request URL	
http://127.0.0.1:8000/headerstructure	
Server response	
Code	Details
200	Response body <pre>{ "html_content": "<!DOCTYPE HTML PUBLIC \"-//W3C//DTD HTML 4.0 Transitional//EN\" \"http://www.w3.org/TR/1998/REC-html40-19980424/loose.dtd\">\n\n<html>\n\n<static/uploaded.png>/>\n\n<meta http-equiv=\"Content-Type\" content=\"text/html; charset=UTF-8\">\n\n<style type=\"text/css\">\n\n<!--\n\n/* character\n000 */ /* default color */\n\n.V { color: #ff0000 } /* duplicate block 1 */\n\n.W { color: #004400 } /* normal block 1 */\n\n.X { color: #ff4488 } /* duplic\n4 */ /* normal block 2 */\n\n.U { color: #cc8844 } /* unused data block */\n\n.H { color: #0000ff } /* highlighted tag name */\n\n.F { color: #aa00dd } /* a\n-decoration: underline */ /* maker notes data */\n\n.tt { /* tooltip text */\n\n visibility: hidden;\n\n position: absolute;\n\n white-space: nowrap;\n\n font-family: Verdana, sans-serif;\n\n font-size: .7em;\n\n padding: 2px 4px;\n\n border: 1px solid gray;\n\n z-index: 3;\n\n}\n\n.tb { /*\n\n lity: hidden;\n\n position: absolute;\n\n background: #ffffff;\n\n zoom: 1;\n\n -moz-opacity: 0.8;\n\n -khtml-opacity: 0.8;\n\n -ms-filter:\n\n t.Alpha(Opacity=80);\n\n filter: alpha(opacity=80);\n\n opacity: 0.8;\n\n z-index: 2;\n\n}\n\n/* table styles */\n\n<table border-top:\n\n 1px solid gray;\n\n}\n\n<table.dump td { padding: .2em .3em }\n\n<td.c2 {\n\n border-left: 1px solid gray;\n\n border-right: 1px solid gray;\n\n}\n\n<table\n\n .9em }\n\n<tbody {\n\n color: black;\n\n background: white }\n\n-->\n\n</style>\n\n<script language=\"JavaScript\" type=\"text/JavaScript\">\n\n<!--\n\n Begin\n\n// too\n\nAR = 4; // top/left margins\n\nvar BMAR = 16; // bottom/right margins (scrollbars may overhang inner dimensions)\n\nvar XOFF = 10; // x offset from cur\n\nvar YMIN = 10; // minimum y offset\n\nvar YTOP = 20; // y offset when above cursor\n\n// common variables\n\nvar safari = navigator.userAgent.indexOf(\n\nnavigator.userAgent.toLowerCase().indexOf('msie 6') >= 0;\n\nvar mspan = new Array;\n\nvar clicked = 0;\n\nvar hlist, tt, tb, firstOutEvt, lastInEvt;\n\n<table\n\n ame, tagname) {\n\n var found = new Array();\n\n var list = document.getElementsByTagName(tagname);\n\n var len = list.length;\n\n for (var i=0, j=0; i\n\n st[i].className.split(' ');\n\n for (var k=0; k<classes.length; ++k) {\n\n if (classes[k] == classname) {\n\n found[j++] = list[i];\n\n }\n\n }\n\n return found;\n\n}\n\n// click mouse\n\nfunction doClick(e)\n\n{\n\n if (!clicked) {\n\n firstOutEvt = lastInEvt = undefined;\n\n high(e, 1\n\n }\n\n else {\n\n clicked = 0;\n\n if (firstOutEvt) high(firstOutEvt, 0);\n\n if (lastInEvt) high(lastInEvt, 1);\n\n }\n\n }\n\n move tooltip\n\n tt)\n\n return;\n\n if (ie6 && (tt.style.top == '' tt.style.top == 0) && (tt.style.left == '' tt.style.left == 0))\n\n {\n\n tt.style.x';\n\n tt.style.height = tt.offsetHeight + 'px';\n\n }\n\n }\n\n var w, h;\n\n // browser inconsistencies make getting window size more complex than it sh\n\n n't know if it is smaller due to scrollbar width\n\n if (typeof(window.innerWidth) == 'number')\n\n {\n\n w = window.innerWidth;\n\n h = window.inn\n\n }\n\n else\n\n {\n\n w = document.documentElement.clientWidth;\n\n h = document.documentElement.clientHeight;\n\n }\n\n }\n\n var x = e.clientX + XOFF;\n\n var y = e.clientY + YOFF;\n\n if (safari) {\n\n patch for p\n\n x -= document.body.scrollLeft + document.documentElement.scrollLeft;\n\n y -= document.body.scrollTop + document.documentElement.scrollTop;\n\n }\n\n h;\n\n var my = h - BMAR - tt.offsetHeight;\n\n if (y > my + YOFF - YMIN) y = e.clientY - YTOP - tt.offsetHeight;\n\n if (x > mx) x = mx;\n\n if (y > my\n\n AR);\n\n if (y < TMAR) y = TMAR;\n\n x += document.body.scrollLeft + document.documentElement.scrollLeft;\n\n y += document.body.scrollTop + document.d</pre>

8. Extract EXIF Metadata

- **Endpoint:** /exif
- **Method:** POST
- **Description:** Upload an image and extract its EXIF metadata.
- **Parameters:**
 - **image:** Image file to upload (multipart form data).
- **Responses:**
 - **200:** EXIF metadata extracted successfully.

```
{
  "Group": [
    ["Description", "Value"],
    ["Description", "Value"],
    ["Description", "Value"].....
  ],

  "Group": [
    ["Description", "Value"],
    ["Description", "Value"],
    ["Description", "Value"].....
  ],
  .....
}
```

- **404:** No EXIF data found.
- **500:** Internal server error.

Example Request:

Curl

```
curl -X 'POST' \  
  'http://127.0.0.1:8000/exif' \  
  -H 'accept: application/json' \  
  -H 'Content-Type: multipart/form-data' \  
  -F 'image=@inputcard1.png;type=image/png'
```

Request URL

```
http://127.0.0.1:8000/exif
```

Server response

Server response

Code

Details

200

Response body

```
{  
  "File": [  
    [  
      "FileCreateDate",  
      "2024:09:09 18:41:07+05:30"  
    ],  
    [  
      "ExifByteOrder",  
      "MM"  
    ],  
    [  
      "ImageWidth",  
      "119"  
    ],  
    [  
      "ImageHeight",  
      "154"  
    ],  
    [  
      "BitsPerSample",  
      "8"  
    ],  
    [  
      "ColorComponents",  
      "3"  
    ],  
  ],  
}
```

Error Codes

- **400 Bad Request:** Invalid request (e.g., missing parameters, incorrect file type).
- **404 Not Found:** The requested resource (header or EXIF metadata) was not found.
- **403 Forbidden:** The user is not authorized to access the requested resource.
- **500 Internal Server Error:** A problem occurred while processing the request.